Database Design Document

1. Introduction

This document outlines the database schema, views, and stored procedures for the Web-based Order Management System (OMS) for **ABC Power Equipment Inc.** The database is designed to be implemented using **PostgreSQL**, catering to order processing, product management, purchase orders, invoicing, and user security.

2. Table Structures

2.1 Users and Authentication

Table: `User`

Stores system user credentials and role-based access information.

Column Name	Data Type	Constraints	Description
`user_id`	UUID / SERIAL	Primary Key	Unique identifier for the user.
`username`	VARCHAR(50)	Unique, Not Null	Login username.
`password`	VARCHAR(255)	Not Null	Hashed password.
`email`	VARCHAR(100)	Unique, Not Null	User's email address.
`role`	VARCHAR(50)	Not Null	Role (e.g., Admin, Sales, Production, Finance).

2.2 Entities (Customers and Suppliers)

Table: `Customer`

Stores customer contact and billing information.

Column Name	Data Type	Constraints	Description
`customer_id`	UUID / SERIAL	Primary Key	Unique identifier for the customer.
`customer_name`	VARCHAR(100)	Not Null	Full name or company name.
`email`	VARCHAR(100)	Unique, Not Null	Contact email.
`phone`	VARCHAR(20)		Contact phone number.
`address`	TEXT		Billing/Shipping address.

Table: `Supplier`

Stores supplier contact information for procurement.

Column Name	Data Type	Constraints	Description
`supplier_id`	UUID / SERIAL	Primary Key	Unique identifier for the supplier.
`supplier_name`	VARCHAR(100)	Not Null	Supplier company name.
`email`	VARCHAR(100)	Unique, Not Null	Supplier contact email.
`phone`	VARCHAR(20)		Supplier phone number.
`address`	TEXT		Supplier physical address.

2.3 Catalog and Inventory

Table: `Product`

Stores information for both finished products and raw components.

Column Name	Data Type	Constraints	Description
`product_id`	UUID / SERIAL	Primary Key	Unique identifier for the product/component.
`product_name`	VARCHAR(150)	Not Null	Name of the product or component.
`description`	TEXT		Detailed description.

Column Name	Data Type	Constraints	Description
`unit_price`	DECIMAL(10,2)	Not Null	Selling or purchasing price.
`quantity_in_stock`	INTEGER	Default 0	Current inventory levels.

Table: `Product_BOM` (Bill of Materials)

Defines the hierarchical relationship between a product and its required components.

Column Name	Data Type	Constraints	Description
`product_bom_id`	UUID / SERIAL	Primary Key	Unique identifier.
`product_id`	UUID / SERIAL	Foreign Key	Maps to `Product(product_id)` (Finished Good).
`component_id`	UUID / SERIAL	Foreign Key	Maps to `Product(product_id)` (Raw Material).
`quantity`	INTEGER	Not Null	Quantity of the component needed.

2.4 Sales Order Management

Table: `Sales_Order`

Records overall sales order details placed by customers.

Column Name	Data Type	Constraints	Description
`sales_order_id`	UUID / SERIAL	Primary Key	Unique order identifier.
`customer_id`	UUID / SERIAL	Foreign Key	Maps to `Customer(customer_id)`.
`order_date`	TIMESTAMP	Default NOW()	Date the order was placed.
`total_amount`	DECIMAL(12,2)	Not Null	Total order value.
`status`	VARCHAR(50)	Not Null	Status (Pending, Processed, Invoiced, Shipped).

Table: `Sales_Order_Item`

Records individual items within a sales order.

Column Name	Data Type	Constraints	Description
`sales_order_item_id`	UUID / SERIAL	Primary Key	Unique item identifier.
`sales_order_id`	UUID / SERIAL	Foreign Key	Maps to `Sales_Order(sales_order_id)`.
`product_id`	UUID / SERIAL	Foreign Key	Maps to `Product(product_id)`.
`quantity`	INTEGER	Not Null	Number of items ordered.
`unit_price`	DECIMAL(10,2)	Not Null	Price per unit at the time of order.

2.5 Purchase Order Management

Table: `Purchase_Order`

Records orders placed to suppliers for procurement.

Column Name	Data Type	Constraints	Description
`purchase_order_id`	UUID / SERIAL	Primary Key	Unique PO identifier.
`supplier_id`	UUID / SERIAL	Foreign Key	Maps to `Supplier(supplier_id)`.
`order_date`	TIMESTAMP	Default NOW()	Date the PO was raised.
`total_amount`	DECIMAL(12,2)	Not Null	Total PO value.
`status`	VARCHAR(50)	Not Null	Status (Draft, Submitted, Received).

Table: `Purchase_Order_Item`

Records individual components within a purchase order.

Column Name	Data Type	Constraints	Description
`purchase_order_item_id`	UUID/SERIAL	Primary Key	Unique PO item identifier.
`purchase_order_id`	UUID / SERIAL	Foreign Key	Maps to `Purchase_Order(purchase_order_id)`.

Column Name	Data Type	Constraints	Description
`product_id`	UUID / SERIAL	Foreign Key	Maps to `Product(product_id)` (Component).
`quantity`	INTEGER	Not Null	Number of components ordered.
`unit_price`	DECIMAL(10,2)	Not Null	Purchasing price per unit.

2.6 Finance (Invoicing & Payments)

Table: `Invoice`

Stores invoice generation details against sales orders.

Column Name	Data Type	Constraints	Description
`invoice_id`	UUID / SERIAL	Primary Key	Unique invoice identifier.
`sales_order_id`	UUID / SERIAL	Foreign Key	Maps to `Sales_Order(sales_order_id)`.
`invoice_date`	TIMESTAMP	Default NOW()	Date invoice was generated.
`total_amount`	DECIMAL(12,2)	Not Null	Amount billed to customer.
`status`	VARCHAR(50)	Not Null	Status (Generated, Sent, Paid, Overdue).

Table: `Payment`

Tracks customer payments made against invoices.

Column Name	Data Type	Constraints	Description
`payment_id`	UUID / SERIAL	Primary Key	Unique payment identifier.
`invoice_id`	UUID / SERIAL	Foreign Key	Maps to `Invoice(invoice_id)`.
`payment_date`	TIMESTAMP	Default NOW()	Date payment was received.
`amount`	DECIMAL(12,2)	Not Null	Amount paid.

Column Name	Data Type	Constraints	Description
`payment_method`	VARCHAR(50)	Not Null	Mode of payment (Card, Bank Transfer, Check).

---

3. Database Views

Views will be implemented to optimize complex reporting and analytics requirements:

1. **`Sales_Order_Details`**:
   - Joins `Sales_Order`, `Sales_Order_Item`, and `Product`.
   - Retrieves comprehensive sales order details including customer info, order timestamps, and product-level breakdowns.
2. **`Purchase_Order_Details`**:
   - Joins `Purchase_Order`, `Purchase_Order_Item`, and `Product`.
   - Retrieves procurement details, vendor information, and requested stock.
3. **`Product_Inventory`**:
   - Joins `Product` and `Product_BOM`.
   - Maps parent products to child components to visualize real-time inventory viability for manufacturing.
4. **`Invoice_Payment_Status`**:
   - Joins `Invoice` and `Payment`.
   - Tracks financial reconciliation, revealing partial payments, unpaid balances, and overdue accounts.

---

4. Stored Procedures / Application Business Logic Functions

As the backend is built on **Spring Boot** with **PostgreSQL**, these procedures can be implemented via PostgreSQL `PL/pgSQL` functions or maintained as transactional services within the Spring Boot application layer.

1. **`Create_Sales_Order(customer_data, products_list)`**
   - Inserts records into `Sales_Order` and `Sales_Order_Item`.
   - Deducts ordered quantities from `Product.quantity_in_stock`.
2. **`Generate_BOM(product_id)`**

- Queries `Product_BOM` recursively (if necessary) for a specific product ID.
- Calculates and returns total required component quantities.

3. **`Create_Purchase_Order(supplier_data, components_list)`**
   - Inserts records into `Purchase_Order` and `Purchase_Order_Item`.
   - Pre-allocates incoming stock status.
4. **`Generate_Invoice(sales_order_id)`**
   - Aggregates the sales order data and creates a new record in `Invoice`.
   - Updates `Sales_Order.status` to "Invoiced".
5. **`Record_Payment(invoice_id, amount, payment_method)`**
   - Inserts a record into the `Payment` table.
   - Validates the total paid against the invoice amount and updates `Invoice.status` to "Paid" if fully settled.
6. **`Generate_Reports(report_type, start_date, end_date)`**
   - Dynamically calls predefined views (`Sales_Order_Details`, `Invoice_Payment_Status`, etc.) based on requested date ranges.
7. **`Manage_Users(user_data)`**
   - Handles administrative CRUD operations on the `User` table, including secure password hashing and role assignments.